

Tytuł: Aproksymacja funkcji ciągłej z użyciem perceptronu dwuwarstwowego

Autorzy: Filip Kucharczyk, Stanisław Gancarczyk

1. Treść i cel zadania

Celem ćwiczenia była implementacja perceptronu wielowarstwowego (MLP) oraz nauczenie go reprezentacji funkcji ciągłej $J: [-5,5] \rightarrow \mathbb{R}$, danej wzorem:

$$J(x) = \sin(x \cdot \sqrt{p[0]+1}) + \cos(x \cdot \sqrt{p[1]+1})$$

gdzie $p=[4, 1]$ to najmłodsze cyfry numerów indeksów autorów. Po podstawieniu wartości, funkcja aproksymowana przyjmuje postać:

$$J(x) = \sin(x \cdot \sqrt{5}) + \cos(x \cdot \sqrt{2})$$

2. Metodyka badań i narzędzia

Badania przeprowadzono przy użyciu autorskiej implementacji sieci neuronowej w języku Python.

- **Architektura sieci:** 1 wejście $\rightarrow$ Warstwa Ukryta (zmienna liczba neuronów) $\rightarrow$ 1 wyjście.
- **Funkcje aktywacji:** Sigmoida (warstwa ukryta), Liniowa (warstwa wyjściowa).
- **Algorytm uczenia:** Stochastyczny Spadek Gradientu (SGD).
- **Miara błędu:** Błąd średniokwadratowy (MSE).

Zgodnie z wymaganiami projektowymi, każdy punkt pomiarowy w tabelach wyników został wyznaczony na podstawie **25 niezależnych uruchomień** algorytmu (dla różnych losowych wag początkowych). Pozwoliło to na wyznaczenie wartości średnich, minimalnych, maksymalnych oraz odchylenia standardowego, co eliminuje wpływ losowości na wnioskowanie.

3. Wyniki eksperymentów

W badaniach skupiono się na analizie wpływu dwóch kluczowych parametrów: liczby iteracji (długości procesu uczenia) oraz liczby neuronów w warstwie ukrytej (pojemności modelu).

3.1. Badanie wpływu liczby iteracji

W pierwszej fazie ustalono stałą, eksperymentalnie dobraną liczbę neuronów ($N=9$) oraz współczynnik uczenia ($LR=0,01$), zmieniając jedynie liczbę epok uczących.

Sieci Neuronowe – Wprowadzenie do sztucznej inteligencji.

Tabela 1. Zestawienie wyników błędu MSE dla zmiennej liczby iteracji (średnia z 25 prób).

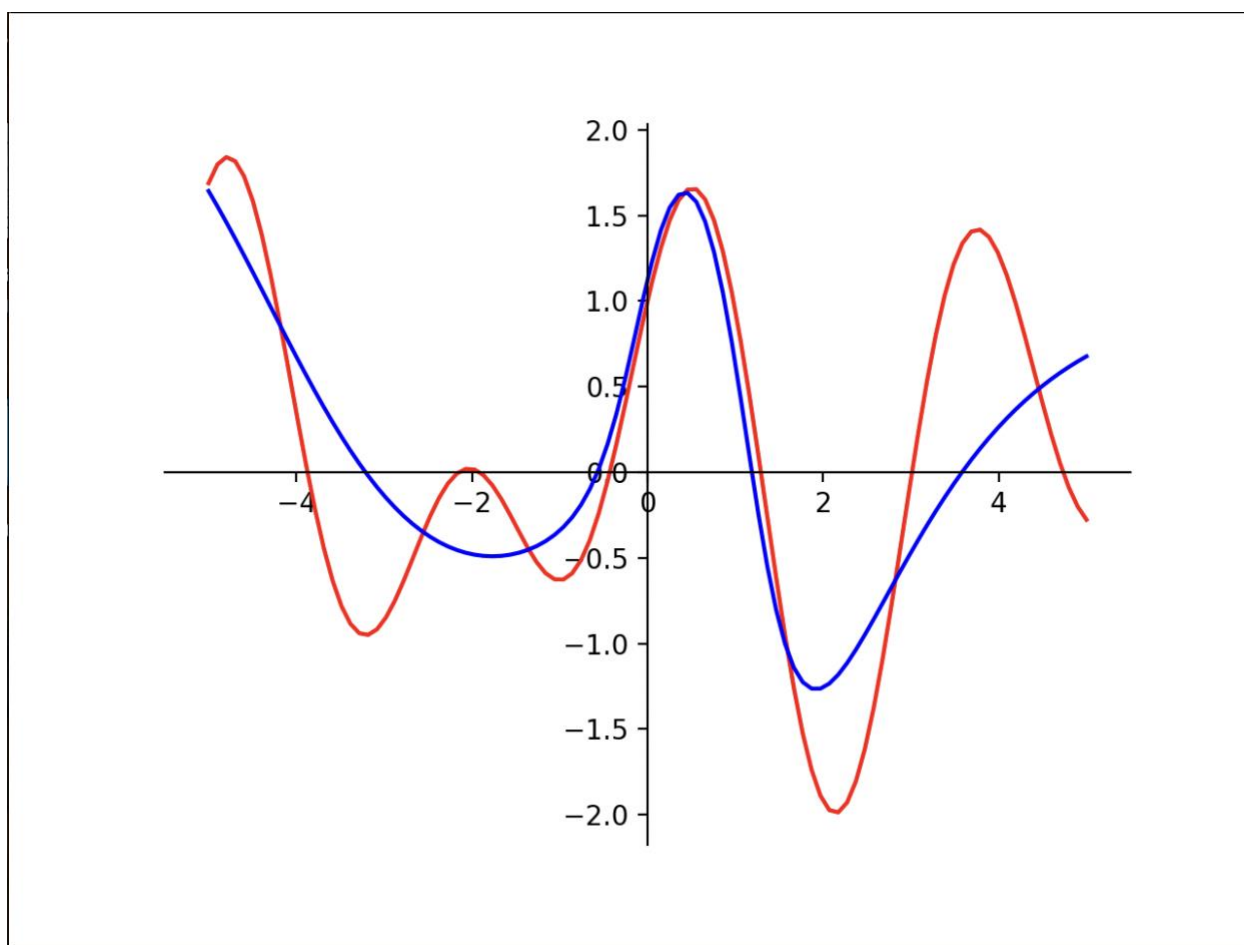
=== BADANIE WPŁYWU PARAMETRU: ITERACJE ===				
Parametry stałe: {'Neurony': 9, 'LR': 0.01}				
Parametr	min	śr	std	max
Iteracje=15000	0,26645	0,33827	0,05109	0,45657
Iteracje=100000	0,02124	0,04062	0,01279	0,07288
Iteracje=300000	0,00146	0,00348	0,00137	0,00673

Analiza wyników:

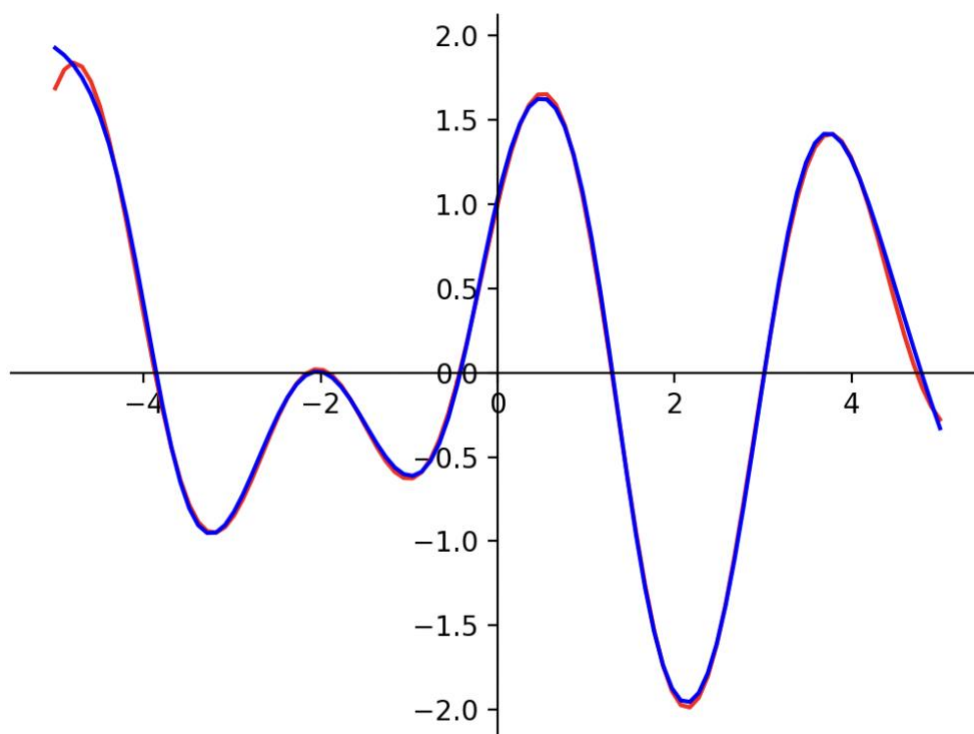
Analiza Tabeli 1 wykazuje silną zależność jakości aproksymacji od czasu uczenia.

- Dla **15 000 iteracji** średni błąd wynosi ok. 0,34. Jest to wynik niesatysfakcjonujący, świadczący o niedouczeniu sieci. Model nie jest w stanie odwzorować pełnej amplitudy funkcji (Rys. 1).
- Zwiększenie liczby iteracji do **300 000** pozwoliło na redukcję błędu o dwa rzędy wielkości (do poziomu ok. 0,003). Jest to najlepszy uzyskany wynik, a sieć niemal idealnie pokrywa się z funkcją celu (Rys. 2).

Wizualizacja wyników:



Rys. 1. Wykres aproksymacji dla 15 000 iteracji. Widoczne znaczne rozbieżności amplitud.



Rys. 2. Wykres aproksymacji dla 300 000 iteracji. Krzywa aproksymacji (niebieska) idealnie pokrywa funkcję celu (czerwoną).

3.2. Badanie wpływu liczby neuronów

W drugiej fazie zbadano wpływ pojemności sieci na wynik uczenia. Ustalono stałą liczbę iteracji na poziomie 100 000 oraz $LR=0,01$.

Tabela 2. Zestawienie wyników błędu MSE dla zmiennej liczby neuronów (średnia z 25 prób).

=== BADANIE WPŁYWU PARAMETRU: NEURONY ===				
Parametry stałe: {'Iteracje': 100000, 'LR': 0.01}				
Parametr	min	śr	std	max
Neurony=3	0,01257	0,04766	0,01701	0,08818
Neurony=9	0,01814	0,04549	0,01896	0,10211
Neurony=20	0,01907	0,04673	0,02314	0,11768
Neurony=50	0,01656	0,03960	0,01748	0,08160

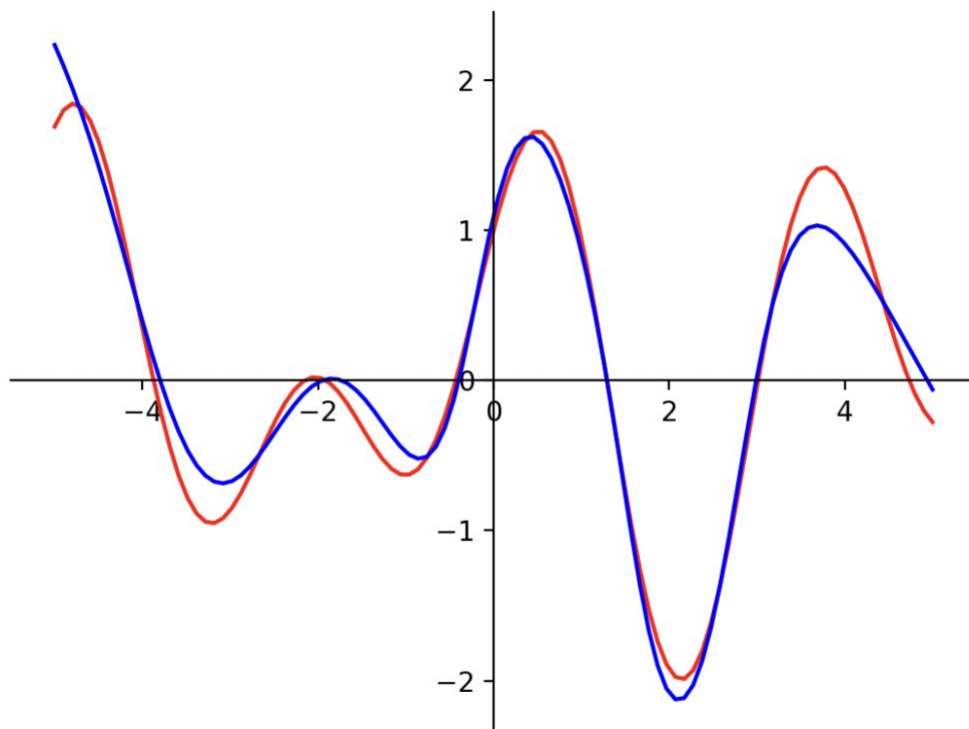
Sieci Neuronowe – Wprowadzenie do sztucznej inteligencji.

Analiza wyników:

Na podstawie danych z Tabeli 2 można sformułować następujące obserwacje:

1. **Nasylenie jakości:** Zwiększenie liczby neuronów z 9 do 20 nie przyniosło poprawy wyniku (średni błąd utrzymał się na podobnym poziomie ok. 0,045-0,046).
2. **Optimum lokalne:** Sieć o mniejszej architekturze (9 neuronów) okazała się łatwiejsza do wytrenowania przy ograniczonej liczbie iteracji niż sieci większe. Wynika to prawdopodobnie z faktu, że przy większej liczbie parametrów (20 lub 50 neuronów) i funkcji aktywacji typu sigmoid, algorytm SGD wolniej zbiega do minimum globalnego.
3. **Ekonomia obliczeniowa:** Najkorzystniejszy stosunek jakości do złożoności obliczeniowej uzyskano dla 9 neuronów.

Wizualizacja wyników:



Rys. 3. Wykres aproksymacji dla dużej liczby neuronów ($N=50$) przy 100 000 iteracji. Mimo dużej pojemności sieci, wynik jest gorszy niż dla $N=9$ przy 300 000 iteracji.

3.3. Badanie wpływu współczynnika uczenia (Learning Rate)

W ostatnim etapie eksperymentów zbadano wpływ parametru *Learning Rate* (współczynnik uczenia) na szybkość i stabilność zbieżności algorytmu. Ustalono stałą architekturę sieci ($N=9$) oraz ograniczoną liczbę iteracji (50 000), aby wyraźnie zaobserwować różnice w tempie uczenia.

Tabela 3. Statystyki błędu MSE w zależności od wartości Learning Rate (LR).

=== BADANIE WPŁYWU PARAMETRU: LR ===				
Parametry stałe: {'Neurony': 9, 'Iteracje': 50000}				
Parametr	min	śr	std	max
LR=0.001	0,55702	0,80935	0,12028	0,97433
LR=0.01	0,09793	0,16140	0,03121	0,24309
LR=0.05	0,00522	0,08485	0,14141	0,75033
LR=0.1	0,00987	0,30090	0,32954	0,93664

Analiza wyników:

Na podstawie Tabeli 3 można wysunąć następujące wnioski dotyczące dynamiki uczenia:

1. **Zbyt mały krok (LR=0,001):** Średni błąd wynosi aż 0,80. Sieć uczy się zbyt wolno i w zadanym czasie (50 tys. iteracji) nie zdążyła zbliżyć się do minimum funkcji kosztu.
2. **Optymalny krok (LR=0,05):** Najlepszy wynik w tym zestawieniu osiągnięto dla LR=0,05. Średni błąd (0,085) jest o połowę mniejszy niż dla wartości domyślnej 0,01. Oznacza to, że przy tej funkcji aktywacji nieco bardziej agresywne uczenie przyspiesza zbieżność.
3. **Granica stabilności (LR=0,1):** Przy zwiększeniu LR do 0,1 średni błąd wzrasta do 0,30, a odchylenie standardowe drastycznie rośnie (do 0,33). Wskazuje to na niestabilność procesu – algorytm wykonuje zbyt duże kroki, "przeskakując" minimum lokalne zamiast w nie trafić.

4. Wnioski końcowe

Przeprowadzone eksperymenty pozwoliły na sformułowanie następujących wniosków dotyczących aproksymacji funkcji perceptronem dwuwarstwowym:

1. **Kluczowa rola liczby iteracji:** W badanym problemie wąskim gardłem nie była pojemność sieci, lecz czas uczenia. Aby uzyskać błąd na poziomie 0,003 przy użyciu funkcji sigmoidalnej, konieczne było wykonanie aż 300 000 kroków (przy LR=0,01).
2. **Dobór współczynnika uczenia (Learning Rate):** Eksperyment wykazał, że dobór odpowiedniego LR jest kluczowy dla efektywności treningu. Dla badanej architektury wartość **LR = 0,05** okazała się optymalna, pozwalając na szybszą redukcję błędu niż standardowe 0,01. Wartości zbyt duże (0,1) prowadziły do destabilizacji procesu uczenia.
3. **Prawo malejących przychodów:** Zwiększanie liczby neuronów powyżej wartości progowej (ok. 9) nie gwarantuje lepszych wyników przy ograniczonej liczbie iteracji, a

Sieci Neuronowe – Wprowadzenie do sztucznej inteligencji.

wręcz może utrudnić proces optymalizacji ze względu na większą liczbę parametrów do wytrenowania.

4. **Optymalna konfiguracja:** Najlepszy stosunek jakości do złożoności obliczeniowej uzyskano dla sieci z **9 neuronami ukrytymi**. Jest to architektura wystarczająca do poprawnego odwzorowania kształtu zadanej funkcji $J(x)$.